Kiste Phase 4.5 Spec

Review, Refactor, and Architecture Checkpoint

One-sentence spec: Kiste Phase 4.5 reviews and stabilizes everything built from Phase 1 to Phase 4, ensuring the repo scanner, token system, multi-repo workspace, GitOps repo, and SOPS secret backend form one clean architecture before adding more advanced deployment features.

Phase 4.5 Position

Phase 4.5 is not a feature expansion phase. It is a checkpoint to review architecture, commands, data models, token handling, GitOps output, and roadmap alignment before Kiste adds heavier deployment, Vault, cloud, registry, or policy-engine features.

Phase	Main focus
Phase 1	Local Docker repo scan + token inspection. Extract app spec, services, ports, env vars, token report, and Kubernetes readiness.
Phase 2	Remote provider scanning for GitHub and Hugging Face, plus GitHub Actions CI testing for the Kiste codebase.
Phase 3	Multi-repo workspace, system dependency graph, and workspace-level token access matrix.
Phase 4	GitOps repo generation, SOPS encrypted secrets, plaintext secret detection, and Vault-ready secret backend design.
Phase 4.5	Review, refactor, stabilize, and confirm the architecture before Phase 5.

1. Why Phase 4.5 Is Needed

- Kiste has expanded from a local repo scanner into a multi-repo, GitOps-aware platform.

- The product now has several major concepts: repo scanning, provider scanning, token inspection, workspace graph, GitOps generation, and secret backends.

- Before adding cloud deployment, Vault, Karmada, cost engines, GPU scheduling, or registry automation, the existing foundation must be cleaned and stabilized.

- Phase 4.5 prevents architecture drift and keeps the MVP realistic.

2. Phase 4.5 Goal

The main goal is to answer whether Kiste is still one coherent product.

```
Is the architecture still clean?
Are commands consistent?
Are data models stable?
Are tokens handled safely?
Are repo, workspace, GitOps, and secret concepts connected correctly?
Is the MVP still realistic?
```

3. Review Areas

A. Command consistency

Kiste should feel like one CLI, not separate prototypes glued together. Phase 4.5 should remove confusing overlaps and stabilize command names.

```
kiste
  scan
  check

  repo
    summary
    services
    env
    ports
    graph

  token
    add
    list
    inspect
    check

  workspace
    init
    add
    list
    scan
    graph
    check
    token

  gitops
    init
    generate
    plan
    validate

  secrets
    init
    add
    scan

  review
    architecture
    commands
    data-model
    security
    gitops
    roadmap
```

B. Data model review

Phase 4.5 should stabilize the core data objects that later phases will depend on.

Object	Purpose
RepoSpec	Normalized result of scanning a local, GitHub, or Hugging Face repo.
WorkspaceSpec	Defines one product/system made from many repos.
Connection	Represents repo-to-repo or service-to-service relationships.
TokenMetadata	Stores provider, scope, expiry, and secret reference, not raw token.

Object	Purpose
TokenAccessResult	Shows what a token can do for a repo or workspace action.
GitOpsSpec	Defines desired GitOps output structure and environment overlays.
SecretBackendConfig	Defines SOPS now, Vault or other backends later.
K8sReadinessReport	Reports what can be generated and what is missing.

C. Token and permission review

Tokens now appear in every major phase. The review must confirm that raw tokens are never stored in plaintext and every token is treated as a scoped capability with expiry.

```
name: github-build-dev
provider: github
scopes:
  - contents:read
  - packages:write
expires_at: 2026-08-01
secret_reference: env://KISTE_TOKEN_GITHUB_BUILD_DEV
```

For Phase 4.5, scan, build, and package permissions should have real checks where possible. Secret and deploy can stay in the model but do not need full implementation yet.

D. GitOps and secret review

Phase 4 introduced GitOps and SOPS. Phase 4.5 should verify that app repos are separated from deployment state, secrets are encrypted, plaintext secret detection can fail the process, and the SecretBackend interface is not hardcoded to SOPS forever.

Design rule: SOPS is the first Git-friendly secret backend. Vault remains a later backend for dynamic runtime secrets, rotation, and centralized policies.

4. Phase 4.5 Commands

Phase 4.5 adds review commands. These commands inspect the project and generate reports, but they do not deploy anything.

```
# Run full review
kiste review

# Review architecture consistency
kiste review architecture

# Review command tree consistency
kiste review commands

# Review generated schemas and object models
kiste review data-model

# Review tokens, permissions, plaintext secrets, and secret backend config
kiste review security

# Review GitOps repo structure and SOPS usage
kiste review gitops

# Review roadmap alignment before Phase 5
kiste review roadmap
```

5. Example Review Output

```
Kiste Phase 4.5 Review

Architecture:
[OK] Repo scanner feeds WorkspaceSpec
[OK] Workspace graph feeds GitOps generator
[OK] Secret backend interface supports SOPS now and Vault later
[WARN] Provider interface needs clearer naming between repo provider and secret backend

Commands:
[OK] scan/check commands are stable
[WARN] token check and workspace token check overlap
[FIX] Keep token check for one token, workspace token check for multi-repo matrix

Security:
[OK] Raw tokens are not stored in workspace files
[OK] SOPS encrypted secrets are allowed
[FAIL] Plaintext Kubernetes Secret files should be rejected before commit

Roadmap:
[OK] Phase 1-4 form a clean pipeline
[WARN] Do not add all cloud providers before GitOps foundation is stable
```

6. Review Output Files

Kiste should generate review reports under .kiste/review/. These files help turn the checkpoint into concrete engineering tasks.

```
.kiste/
  review/
    architecture-review.md
    command-review.md
    data-model-review.md
    security-review.md
    gitops-review.md
    roadmap-review.md
    kiste-phase-4.5-review.md
```

7. Architecture Review Checklist

Area	Question	Expected result
Repo scanner	Can local, GitHub, and Hugging Face repos feed the same scanner pipeline?	[OK] One normalized provider interface.
Workspace graph	Can scanned repo specs become one system graph?	[OK] WorkspaceSpec and Connection models are stable.
Token system	Can Kiste prove repo/action access before expiry?	[OK] Token access matrix works.
GitOps generator	Can the graph generate a deployment repo structure?	[OK] GitOpsSpec is derived from WorkspaceSpec.
Secret backend	Can SOPS work now without blocking Vault later?	[OK] SecretBackend interface exists.

8. Security Review Checklist

Check	Pass condition
Raw token storage	No raw tokens in kiste.yaml, workspace.json, token-matrix.json, GitOps repo, or logs.
Token expiry	Every token has expires_at metadata and warns before expiry.
Token scope	Kiste maps internal actions to provider-specific permissions.
Plaintext secrets	Plaintext Kubernetes Secret manifests and .env files are rejected in GitOps repo.
SOPS files	Only encrypted *.enc.yaml secret files are allowed under secrets/ by default.
Vault readiness	SecretBackend interface is generic enough for Vault later.

9. Roadmap Review

Phase 4.5 should decide what must be fixed before Phase 5. The priority is not to add every cloud, provider, or registry immediately. The priority is to keep Kiste understandable and implementable.

Item	Recommendation
Cloud deployment	Delay until GitOps output and secret handling are stable.
Vault	Keep as later backend; do not replace SOPS in Phase 4.5.
Karmada	Delay until single-cluster GitOps flow is clean.
Cost/GPU engine	Keep data fields, but delay full optimizer.
More providers	Add only after provider abstraction is tested well.
Container registries	Important later, but not before GitOps structure and image linkage are designed cleanly.

10. What Phase 4.5 Should Not Do

- No real Kubernetes deployment.

- No Terraform apply.

- No full Vault integration.

- No dynamic database credentials.

- No Karmada multi-cluster scheduling.

- No full cost optimizer.

- No full GPU scheduler.

- No provider explosion before abstractions are stable.

11. Acceptance Criteria

Phase 4.5 is complete when the review workflow can run and produce actionable reports.

```
kiste workspace scan
kiste workspace graph
kiste gitops validate
kiste secrets scan
kiste review
```

Required outputs:

- Architecture review.

- Command tree review.

- Data model review.

- Token and security review.

- GitOps/SOPS review.

- Roadmap review.

- List of issues to fix before Phase 5.

12. Final Phase 4.5 Summary

Phase 4.5 is the checkpoint that turns Kiste from a set of useful ideas into a maintainable platform architecture. It protects the core design: scan repos, build a workspace graph, check tokens, generate GitOps desired state, encrypt secrets, and keep future backends such as Vault possible without rewriting the system.